

基于 Zeek 流量与 UEBA 的防泄密产品设计方案

1. 总体架构

本方案采用“端-网-云”一体化设计，通过采集网络流量元数据和 Linux 主机行为日志，经清洗、标准化、富化后存入分析引擎，利用孤立森林（Isolation Forest）等算法实现用户与实体异常行为检测。

架构分为采集层 → 传输层 → 处理层 → 存储层 → 分析层 → 可视化/告警层，各层松耦合，可水平扩展。

2. 数据源与采集

数据源类型	具体日志/数据
网络流量	Zeek 生成的 <code>conn.log</code> , <code>http.log</code> , <code>dns.log</code> , <code>ssl.log</code> 等
Linux 主机	<code>/var/log/auth.log</code> （认证）、 <code>/var/log/audit/audit.log</code> （审计）、 <code>/var/log/secure</code> （安全）
应用/服务	Nginx/Apache 访问日志、MySQL 审计日志
身份与网络	LDAP/AD 认证日志、防火墙日志、VPN 日志

3. 详细设计步骤与技术栈

步骤	关键操作	输入日志
① 采集	每台主机部署采集器，实时读取日志文件并发送至消息队列	原始文本化
② 传输	消息队列缓冲，削峰填谷，保证高吞吐与可靠性	采集器输

5. 关键选型与注意事项

组件	推荐版本/方案	注意事项
Zeek	5.0+	需配置合适的 <code>networks.cfg</code>
Zeek Agent	实验性，建议测试	若投入生产，优先用 Filebeat
Filebeat	7.x/8.x	启用 <code>close_renamed</code> 和 <code>close</code>
Kafka	2.8+	分区数 $\geq$ 节点数，副本数 ≥ 2
Logstash	7.x/8.x	使用 <code>pipeline.workers</code> 和 <code>b</code>
Elasticsearch	7.x/8.x	每日新建索引（如 <code>logs-YYYY.</code>
孤立森林模型	Scikit-learn 1.0+	特征工程需覆盖：登录频次、
告警	ElastAlert 或 Kibana Alerting	避免告警风暴，设置静默期和

6. 落地实施建议

1. 试点先行：先选择 10~20 台测试服务器，部署完整链路，验证数据质量和检测效果。
2. 基线建立：运行 1~2 周，收集正常行为数据，用于初始模型训练。
3. 模型调优：根据误报/漏报率调整孤立森林的 `contamination` 参数和阈值。
4. 数据安全：传输全程 TLS 加密，存储敏感字段（如密码）脱敏或哈希。
5. 高可用设计：Kafka、ES 都需集群部署，避免单点故障。
6. 性能评估：单台 Filebeat 内存约 50~100MB，Logstash 按处理量横向扩展，ES 至少 3 节点。

7. 总结

本方案以 **Zeek** 提供网络数据，以 **Filebeat + Kafka + Logstash** 完成主机日志采集与清洗，以 **Elasticsearch** 统一存储，以 **孤立森林模型**（Python 服务）进行实时异常检测，最

终通过 **Kibana** 展示并触发告警。各组件均为成熟开源方案，工程落地可行性高，只需按上述规划准备人力（运维、开发、算法）和硬件资源即可。

8. 补充材料

为上千台Linux环境构建UEBA系统，是一个典型的大数据工程问题。它的核心在于建立一个从分散、非结构化数据，到集中、结构化、可供分析的数据处理管道。

📊 第一步：UEBA的日志从哪来？（数据源）

UEBA的精髓在于多源数据关联分析。对于Linux环境，日志主要分为以下几类：

- **系统与安全日志（核心）**：这是最基础的数据。
 - **认证与授权**：`/var/log/auth.log` 或 `/var/log/secure`，记录所有登录（SSH、本地）、`sudo`、`su` 等。
 - **系统审计**：`/var/log/audit/audit.log`，由 `auditd` 生成，记录细粒度的系统调用、文件访问等。
 - **系统运行**：`/var/log/syslog` 或 `/var/log/messages`，记录系统级非内核的通用事件。
 - **内核与硬件**：`/var/log/kern.log` 和 `dmesg`，记录内核及硬件相关信息。
 - **失败登录**：`/var/log/faillog`，专门记录失败的登录尝试。
 - **系统日志**：`systemd-journald`，可用 `journalctl` 命令访问。
- **应用与服务日志**：业务系统的“操作手册”。
 - **Web服务器**：如Nginx、Apache的访问日志和错误日志。
 - **数据库**：如MySQL、PostgreSQL的审计日志和查询日志。
 - **企业应用**：如业务系统日志、消息队列（MQ）日志等。
- **网络与安全设备日志**：内外交互的“监控录像”。
 - **网络流量**：通过Zeek等工具产生的流数据（Flow Data）。
 - **安全设备**：防火墙、入侵检测/防御系统（IDS/IPS）、VPN、堡垒机、数据防泄漏（DLP）等设备的日志。
- **身份与访问管理日志**：确定“谁”在操作的关键。
 - **认证系统**：如LDAP、Active Directory（AD）、单点登录（SSO）系统的认证日志。
 - **代理与网关**：内部网络的代理服务器、API网关的访问日志。

🔧 第二步：如何从上千台服务器收集日志？（采集层）

你需要一个轻量级、高性能、高可靠的日志采集器，部署在每台服务器上。常见方案包括：

1. **Fluentd / Logstash**: 功能强大的日志管道工具，支持丰富的输入、过滤和输出插件。Logstash是ELK技术栈的一部分，而Fluentd更轻量。
2. **Filebeat (Elastic Stack)**: Elastic公司推出的轻量级日志采集器，资源占用小，配置简单，非常适合于大规模采集场景。
3. **Rsyslog**: 大多数Linux发行版默认的日志服务，性能极高，能直接通过简单配置将系统日志转发到中央服务器。
4. **Zeek Agent**: 如果已在网络层部署了Zeek，用它来采集主机日志，能实现网络和主机数据的天然关联。

⚠ 关键挑战与对策:

- **可靠性**: 必须确保日志不丢失。优先选用支持**TLS加密**和**TCP协议**（有确认重传机制）的传输方式。
- **性能影响**: 采集器对宿主机的CPU和内存占用必须极低，避免影响业务。**Filebeat**是这方面的首选。
- **配置管理**: 上千台服务器的采集器配置必须自动化、可编排，建议使用**Ansible**、**Puppet**等工具统一管理。
- **缓冲机制**: 当中央存储不可用时，采集器应有本地缓冲（如Filebeat的 `spool` 机制），防止数据丢失。

📦 第三步：海量日志如何存放？（存储层）

日志数据是典型的写多读少、冷热分明的场景，需要分层存储策略。

- **热存储（实时查询与分析）**: 用于最近（如30天内）的日志，要求写入快、查询快。
 - **Elasticsearch**: 最主流的日志存储方案，提供强大的全文检索和聚合分析能力。
 - **ClickHouse / Apache Doris**: 专为OLAP设计的列式数据库，在超大规模（PB级）日志场景下查询性能卓越。
- **温/冷存储（长期归档与合规）**: 用于存放超过30天的历史日志，要求存储成本低。
 - **对象存储**: 如Amazon S3、阿里云OSS，成本极低。**Elasticsearch**等支持将旧索引自动冻结或迁移到对象存储。
- **存储设计关键点**:
 - **分片与分区**: 按时间进行分区/分片是必须的，能极大提升查询效率并便于数据生命周期管理。
 - **索引优化**: 仅为需要快速检索的字段建立索引，平衡写入和查询性能。
 - **数据压缩**: 使用列式存储和高效的压缩算法（如ZSTD）来节省存储空间。

🔪 第四步：杂乱日志如何变得可用？（清洗与预处理层）

原始日志是“非结构化”的文本，必须清洗成“结构化”数据才能进行分析。

- **格式解析**：使用Grok模式等规则，从非结构化文本中提取出关键字段。例如，将 `sshd[12345]: Failed password for root from 192.168.1.100 port 22 ssh2` 解析为 `{ "service": "sshd", "event": "failed_password", "user": "root", "src_ip": "192.168.1.100" }`。
- **数据标准化**：统一不同来源的字段名。例如，将所有表示“源IP”的字段（`src_ip`，`client_ip`，`remote_addr`）统一为 `source_ip`。
- **数据富化**：通过关联外部数据为日志增加上下文。例如，将IP地址关联地理位置，将用户名关联组织架构和部门，将进程名关联威胁情报。
- **数据过滤与归并**：过滤掉明显无用的信息（如健康检查心跳），或将重复的日志进行聚合，以减少存储和分析压力。

推荐工具：**Logstash**或**Fluentd**的过滤器（Filter）组件是完成上述任务的理想选择。

🧠 第五步：处理好的日志如何分析？（分析层）

这是UEBA的核心，孤立森林算法就在这里发挥作用。

1. **行为基线建立**：基于清洗好的历史数据（如过去30天），为每个用户、主机、IP等“实体”建立正常行为基线。
2. **实时异常检测**：将实时流入的日志与基线对比。孤立森林等算法非常适合在此阶段识别出“异常点”。
3. **多源关联分析**：将来自网络层（Zeek）的异常连接，与来自主机层（审计日志）的进程创建日志关联。例如，发现 `host A` 在凌晨3点通过 `wget` 下载了一个脚本，随后Zeek检测到 `host A` 向一个罕见IP发起了外连。这种跨数据源的关联才是UEBA的真正价值所在。
4. **风险评分与告警**：对检测到的异常事件进行风险评估和打分，并按风险等级进行告警。

技术选型：通常，清洗后的日志会存入**Elasticsearch**，你可以利用其原生支持的机器学习功能进行异常检测，或者，更灵活的方式是：从**Elasticsearch**中读取数据，交由一个独立的**Python/Java**算法服务（如你的孤立森林模型）进行分析，再将结果写回Elasticsearch用于展示。

💎 总结：端到端的日志分析架构

将以上所有步骤串联起来，就是一个典型的、可用于UEBA的日志分析架构：

- **采集层**：在每台服务器上部署 **Filebeat**。
- **传输层**：Filebeat将日志安全地传输到 **Kafka** 消息队列，以应对流量高峰，起到削峰填谷的作用。
- **处理层**：**Logstash** 或 **Fluentd** 从Kafka消费数据，进行解析、标准化和富化。
- **存储层**：处理后的结构化数据存入 **Elasticsearch** 集群。
- **分析层**：
 - **在线分析**：你的孤立森林算法服务从Kafka或Elasticsearch获取实时数据进行检测。
 - **离线分析**：利用 **Spark** 等大数据引擎对Elasticsearch中的历史数据进行批处理分析，用于模型训练和基线更新。

- **可视化层**：通过 **Kibana** 进行日志搜索、可视化展示和告警配置。

这个架构复杂度不低，但每一个组件都是久经考验的开源方案，能支撑起上千台服务器的日志处理需求。它为你提供了一个坚实的基础，让你可以专注于最核心的**异常检测算法和威胁分析**。

参考：

<https://www.secrss.com/articles/7252>

<https://www.secrss.com/articles/12723>

<https://blog.csdn.net/PearlOwl67/article/details/156832937#1>